



Building Regression Models



Dr. Gagan Raj Gupta, CSE, IIT Bhilai

Lecture Objectives

- What we will learn in this lecture?
 - Regression Analysis and examples
 - Machine Learning Methodology
 - How to assess goodness of the models
 - Understand the bias-variance trade-off
 - Understand the method of K nearest neighbors
 - Understand the method of linear (least-squares) regression
 - Polynomial regression
 - Introduction to Neural Networks

Prediction problem motivation

- The variable we'd like to predict may be:
 - more difficult to measure
 - is more important than the other(s)
 - or may be directly or indirectly influenced by the values of the other variable(s)
- Thus, we'd like to define two categories of variables:
 - variables whose value we want to predict
 - variables whose values we use to make our prediction

Regression Analysis

- **Definition:** A class of techniques that seeks to make predictions about unknown continuous target variables given observed input variables.
- **Applications:**
 - Predicting a person's height given the height of their parents.
 - Predicting the amount of time someone will take to pay back a loan given their credit history.
 - Predicting what time a package will arrive given current weather and traffic conditions.
 - Predicting the production of a particular crop given the rainfall

Response and Predictor Variables

- We are observing $p + 1$ numerical variables and we are making n sets of observations.
- We call the variable we'd like to predict the **outcome** or **response variable**; typically, we denote this variable by Y and the individual measurements y_i .
- The variables we use in making the predictions the **features** or **predictor variables**; typically, we denote these variables by $X = X_1, \dots, X_p$ and the individual measurements $x_{i,j}$.

Note: i indexes the observation ($i = 1, \dots, n$) and j indexes the value of the j -th predictor variable ($j = 1, \dots, p$). Total number of predictor variables, $J=p$

True vs. Statistical Model

- We will assume that the response variable, Y , relates to the predictors, X , through some unknown function 'f' expressed generally as:

$$Y = f(X) + \varepsilon$$

- Here, f is the unknown function expressing an underlying rule for relating Y to X , ε is the random amount (unrelated to X) that Y differs from the rule $f(X)$.
- A **statistical model** is any algorithm that estimates f . We denote the estimated function as $\hat{f}$.

Machine Learning

Data Science Process

Ask an interesting question

Data preparation

Explore the Data

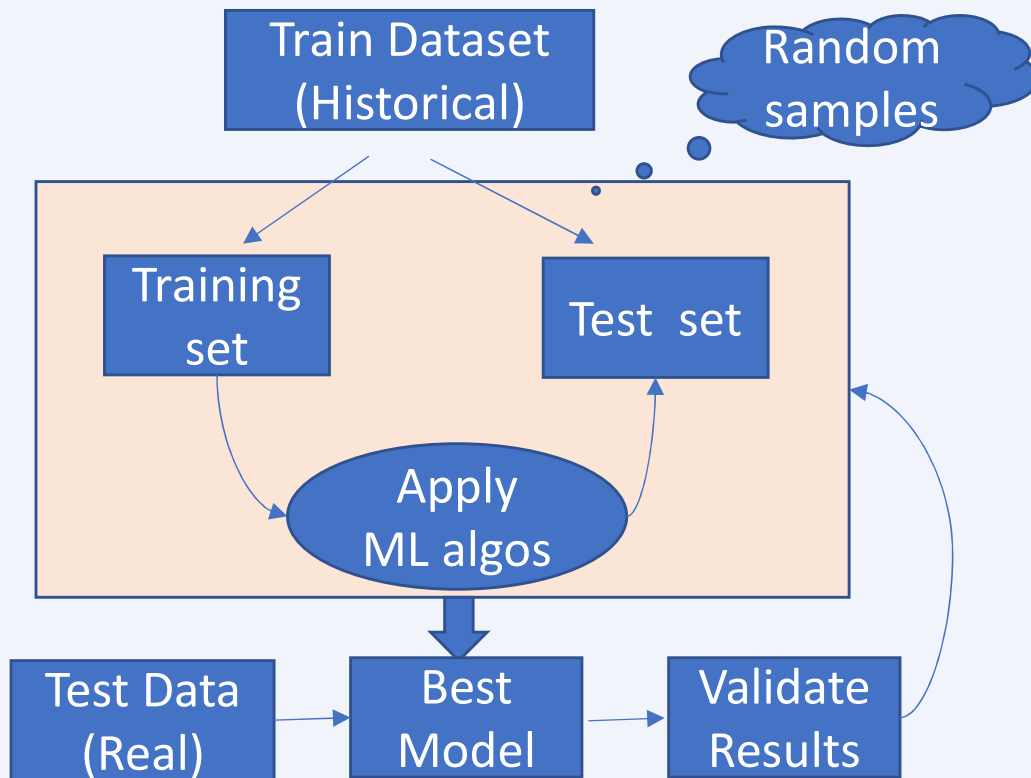
Model the Data

Communicate/Visualize the Results

- **ML algorithms have the objective of generalization**, i.e. use one dataset to generate models that perform well on data that they have not seen.
- Thus, they prove to be effective in generating predictive models.
- There are many ML techniques and ML models have several parameters
- How to choose the best one?



Machine Learning Methodology



- Input dataset is divided into a random split (80/20) or (90/10) to be used for training and testing respectively
- For each split, model is generated and tested for accuracy
- This is repeated 5 times or 10 times and average error is computed
- The best model is selected and used in real world scenario

Flexibility vs. Interpretability Tradeoff

- There are many methods of regression (that estimate f)
- Some are less flexible but more interpretable
- These are useful for inference problems where we want to study the relationships between predictor variables
- But highly flexible methods can also lead to over-fitting!



Error Evaluation

In order to quantify how well a model performs, we define a *loss* or *error function*.

A common loss function for quantitative outcomes is the **Mean Squared Error (MSE)**:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

The quantity $y_i - \hat{y}_i$ is called a *residual* and measures the error at the i -th prediction.

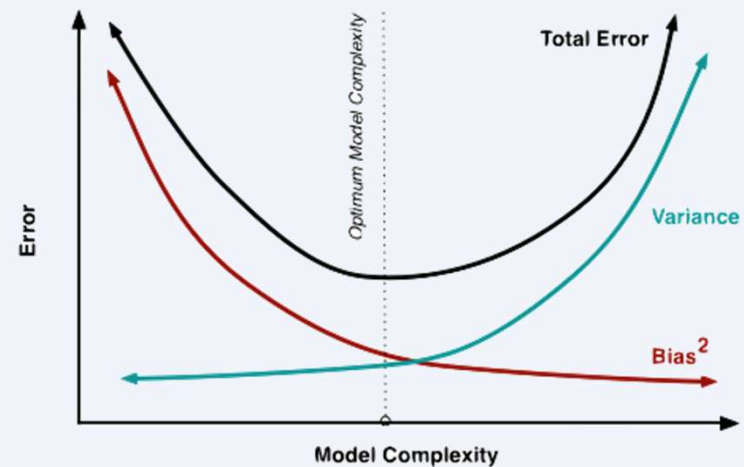
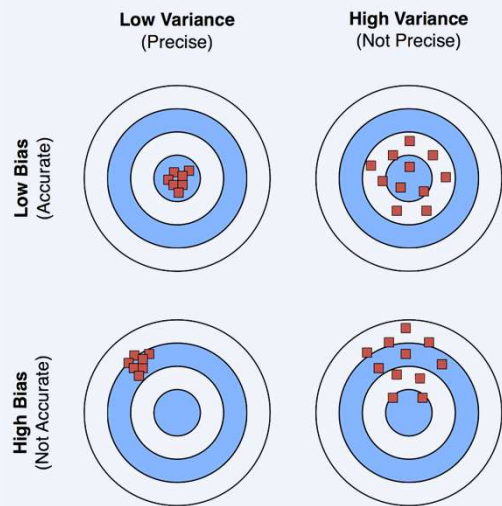
The square root of MSE is RMSE: $RMSE = \sqrt{MSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$

R-squared Error

$$R^2 = 1 - \frac{\sum_i (\hat{y}_i - y_i)^2}{\sum_i (\bar{y} - y_i)^2}$$

- If our model is as good as the mean value, $\bar{y}$, then $R^2 = 0$
- If our model is perfect then $R^2 = 1$
- R^2 can be negative if the model is worse than the average. This can happen when we evaluate the model in the real life test set.

Bias Variance Tradeoff



- Total Error = Bias² + Variance + Irreducible Error

Bias is the average distance of estimate $\hat{f}$ from the true mean of $f(x)$

Variance is the sq. dev of the estimate $\hat{f}$ around its mean

Bias Variance Tradeoff

“All models are wrong, but some models are useful.” : George Box (1919-2013)

- Occam's razor: This philosophical principle states that “the simplest explanation is best”.
- **Bias** is error from erroneous assumptions in the model, like making it linear/simplistic. (underfitting)
- **Variance** is error from sensitivity to small fluctuations in the training set, indicating it will not work in real world. (overfitting)
- First-principle models likely to suffer from bias, with data-driven models in greater danger of overfitting.

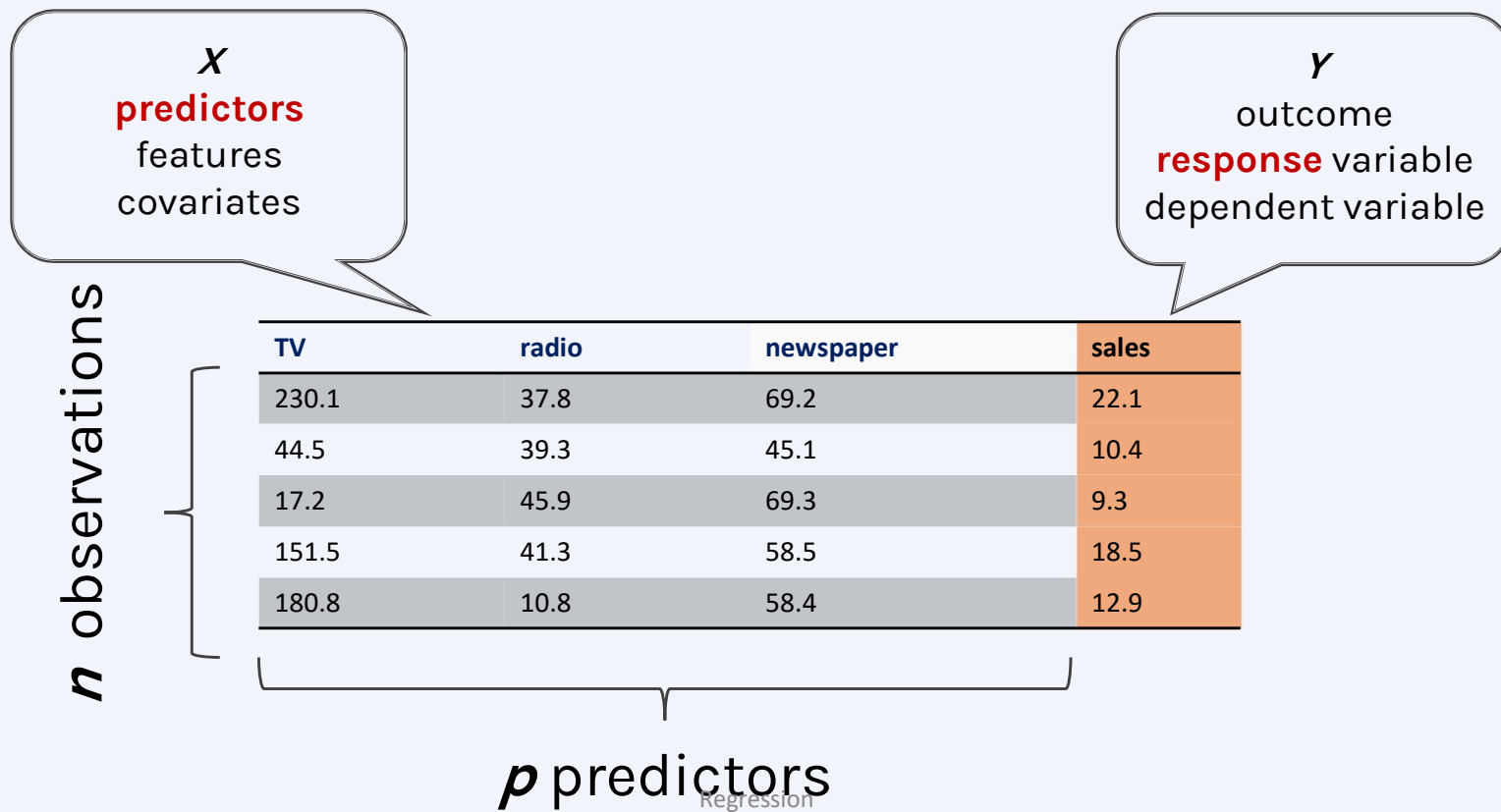
Example Problem (Advertising)

TV	radio	newspaper	sales
230.1	37.8	69.2	22.1
44.5	39.3	45.1	10.4
17.2	45.9	69.3	9.3
151.5	41.3	58.5	18.5
180.8	10.8	58.4	12.9

The **Advertising data set** consists of the sales of that product in 200 different markets, along with advertising budgets for the product in each of those markets for three different media: TV, radio, and newspaper. Everything is given in units of \$1000.

Some of the figures in this presentation are taken from ISL book: "An Introduction to Statistical Learning, with applications in R" (Springer, 2013) with permission from the authors: G. James, D. Witten, T. Hastie and R. Tibshirani "

Response vs. Predictor Variables



k-Nearest Neighbors

The ***k-Nearest Neighbor (kNN) model*** is an intuitive way to predict a quantitative response variable:

to predict a response for a set of observed predictor values, we use the responses of other observations most similar to it

Note: this strategy can also be applied in classification to predict a categorical variable.

k-Nearest Neighbors

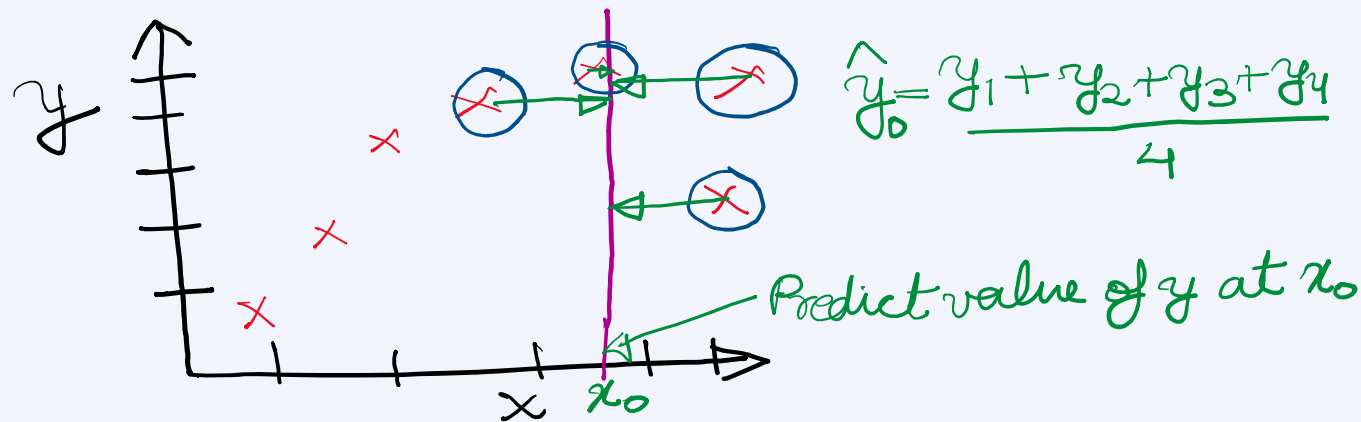
For a fixed a value of k , the predicted response for the i -th observation as the average of the observed response of the k -closest observations:

$$\hat{y}_n = \frac{1}{k} \sum_{i=1}^k y_{n_i}$$

where $\{x_{n1}, \dots, x_{nk}\}$ are the k observations most similar to x_i (*similar* refers to a notion of distance between predictors). Usually, Euclidean distance is chosen (sqrt of squared coordinate differences)

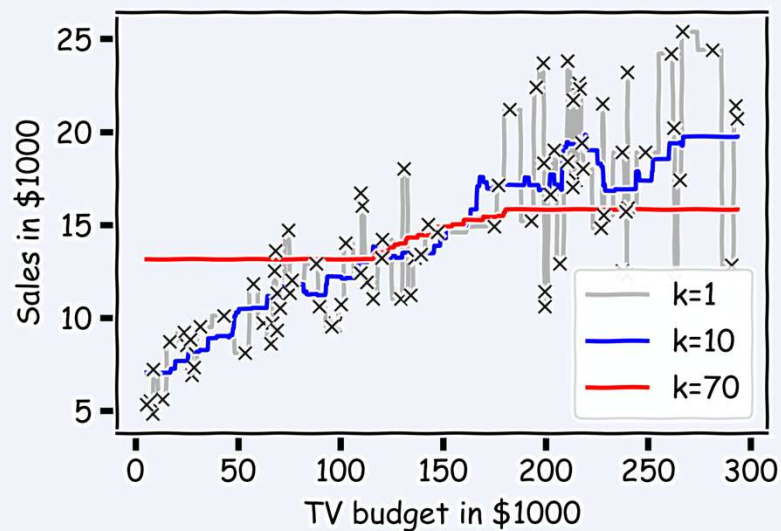
```
Python: sklearn.neighbors.KNeighborsRegressor(n_neighbors=3)
```

4-Nearest Neighbors

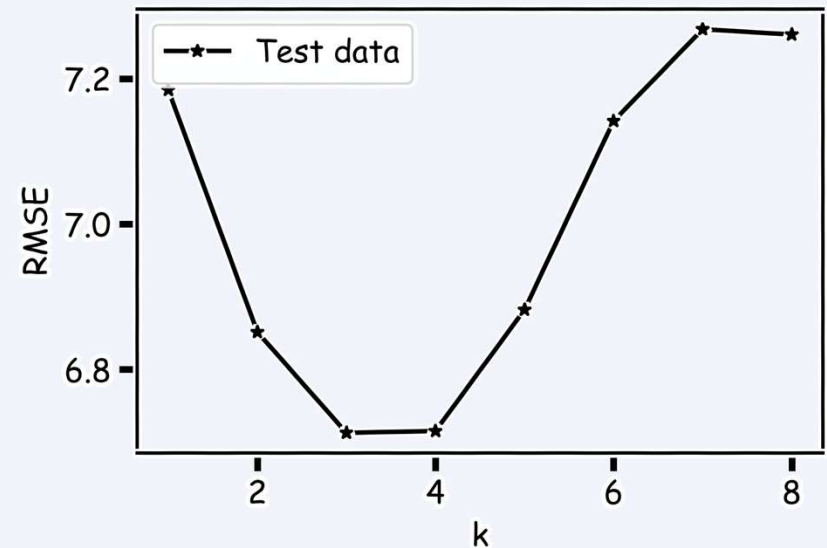


- Very few assumptions made here about the nature of 'f'
- Equal weights for the values of y , regardless of their distance from x
- As dimensionality increases, becomes hard to find neighbors closeby and f may change significantly

Model Comparison



Q. Reason for discontinuity
Q. Bias Variance tradeoff



Do the same for all k 's and compare the RMSEs
Which k is best?

Linear Models

- In the kNN approach, we didn't assume a form of the function 'f'
 - Such approaches are called non-parametric approaches
- In the linear regression approach, we assume that the response is a linear function of the predictor variables
- Note that this technique can be easily extended by creating extra predictor variables (features) from a combination (transformation) of the original predictor variables.
- So let's assume:

$$Y = f(X) + \epsilon = \beta_1 X + \beta_0 + \epsilon.$$

Linear Regression

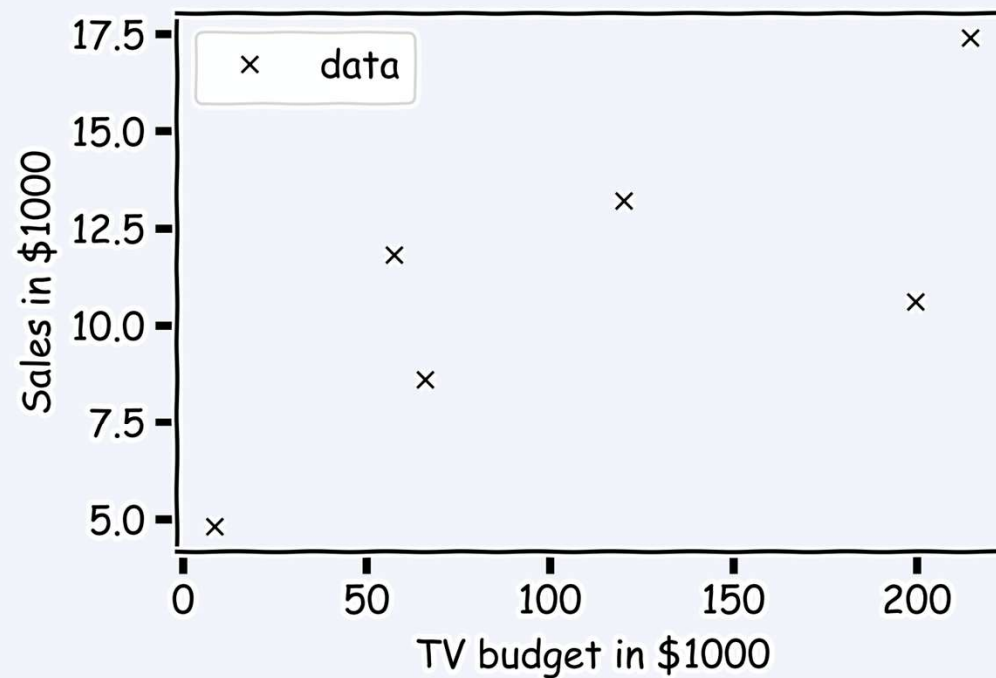
- ... then it follows that our estimate is:

$$\hat{Y} = \hat{f}(X) = \hat{\beta}_1 X + \hat{\beta}_0$$

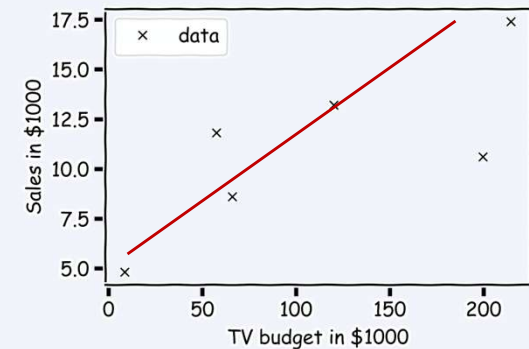
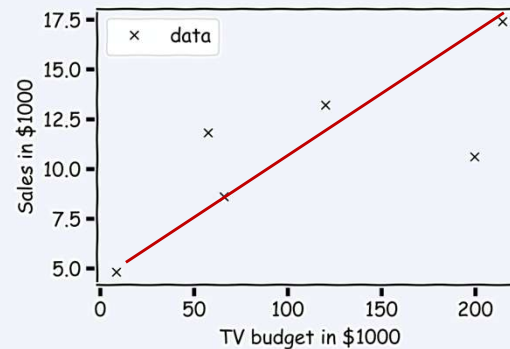
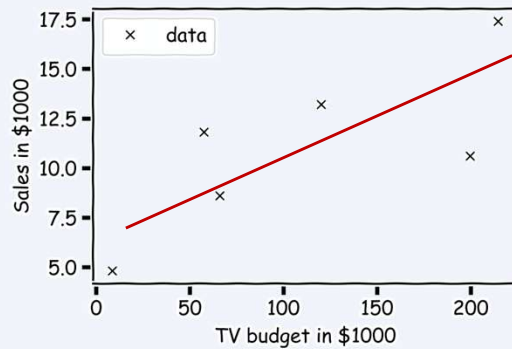
- where $\hat{\beta}_1$ and $\hat{\beta}_0$ are **estimates** of β_1 and β_0 respectively, that we compute using observations.

Estimate of the regression coefficients

For a given data set



Estimate of the regression coefficients (cont)

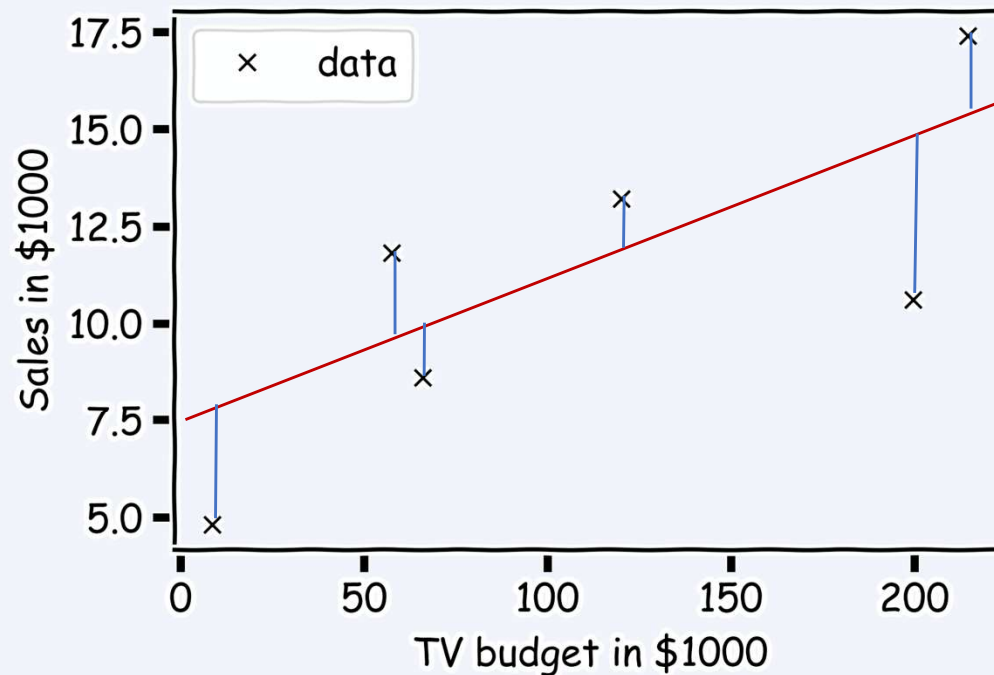


Which of the above three lines fit the data points the best?

- a. One which goes through maximum number of points
- b. One with least slope
- c. One from which no point is too far, i.e. it is approximately in middle of all points

Estimate of the regression coefficients (cont)

To compute the best fit, we first calculate the residuals



Python package

```
X=market_data[['TV','radio','newspaper']]
Y=market_data['sales']
model= sklearn.linear_model.LinearRegression()
model.fit(X,Y)
```

```
LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None, normalize=False)
```

```
X_new=[[147,23,30.55]]
print(model.predict(X_new))
```

```
[13.97078723]
```

So how do Linear Regression solvers work?

- Matrix Methods
 - Exact methods that solve the set of linear equations
 - Involve computation of matrix inverse or pseudoinverse (more efficient)
- Gradient Descent
 - A generic method of solving optimization problems
 - Begin with a random point and reach the optimal solution through a sequence of improvements
 - Faster improvements could be done by stochastic methods

Matrix Algebra for n-dimensions

- Loss (L) = $\text{MSE}(\beta) = \sum_{i=1}^N (y_i - \mathbf{x}_i^T \beta)^2$
- $\text{MSE}(\beta) = (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta)$, where $\mathbf{X}$ is a $n \times (p+1)$ matrix with each row as an input vector (including '1' for the intercept) and $\mathbf{y}$ is a n dimensional vector of the outputs in the training set
- To minimize, we differentiate with respect to β and we get
 - $(\mathbf{X})^T (\mathbf{y} - \mathbf{X}\beta) = 0$
- If $\mathbf{X}^T \mathbf{X}$ is non-singular, meaning, inverse exists, then
 - $\beta = (\mathbf{X}^T \mathbf{X})^{-1} (\mathbf{X}^T \mathbf{y})$

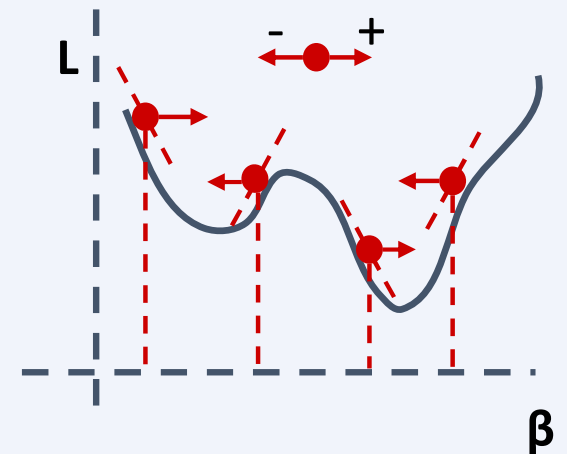
Matrix Algebra for n-dimensions

- Computational complexity of computing the matrix inverse: $O(p^{2.4})$ to $O(p^3)$ depending on implementation.
- Scikit-learn's Linear Regression class uses SVD approach $O(p^2)$
 - SVD stands for singular value decomposition
 - Uses pseudoinverse approach (Moore-Penrose) : `numpy.linalg.pinv()`
 - $\beta = (X^+ y)$
- They both have linear complexity in terms of the number of instances, n ; but at least quadratic in p
- So, we need to look at alternate techniques if p is very large, e.g., 100,000

Gradient Descent Approach

- Start from a random point, i.e. generate a random β
 1. Determine which direction will reduce the MSE
 2. Compute the slope of the function (its derivative) at this point and go in the reverse direction
 3. λ is the learning rate parameter
 4. Go to #1, until convergence, i.e. MSE is minimized
- For Linear regression, MSE is a convex function
 - There is no local minima, just a global minimum
 - Continuous and slope that never changes abruptly

$$\beta^{(i+1)} = \beta^{(i)} - \lambda \frac{dL}{d\beta}$$



Stochastic Approaches

- Batch GD update equation $\boldsymbol{\beta}^{(i+1)} = \boldsymbol{\beta}^{(i)} - \lambda \frac{2}{p} (\mathbf{X})^T (\mathbf{X}\boldsymbol{\beta} - \mathbf{Y})$
 - Uses the whole batch of training data at each gradient step
- Stochastic GD
 - Picks only a random instance of training data to update gradients
 - Causes irregular descent, but better chance of finding global minimum
 - Simulated annealing: Reduce the learning rate gradually to reduce irregularity
- Mini-batch GD
 - Small set of random instances of training data are used

Parametric or Non-Parametric?

	Linear Regression (parametric)	k-NN Approach (non-parametric)
Assumption on function f	A linear function is assumed	Can work even if the function is non-linear. But it has to be locally constant
High dimensions	Complexity problems which can be overcome by efficient algorithms	Difficult to find neighbors nearby which can cause errors
Bias	Low	Small $K \Rightarrow$ Low bias Large $K \Rightarrow$ High Bias
Variance	Depends on the problem	Small $K \Rightarrow$ High Variance Large $K \Rightarrow$ Low Variance
Computations	Once during the model fitting phase. After that predictions are quick	Every time a prediction has to be made, we look at all the training points

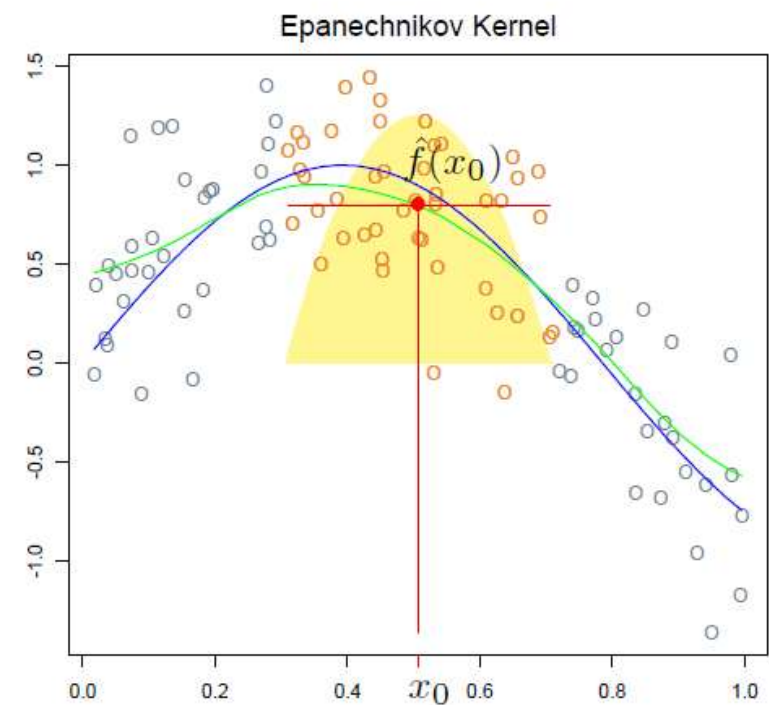
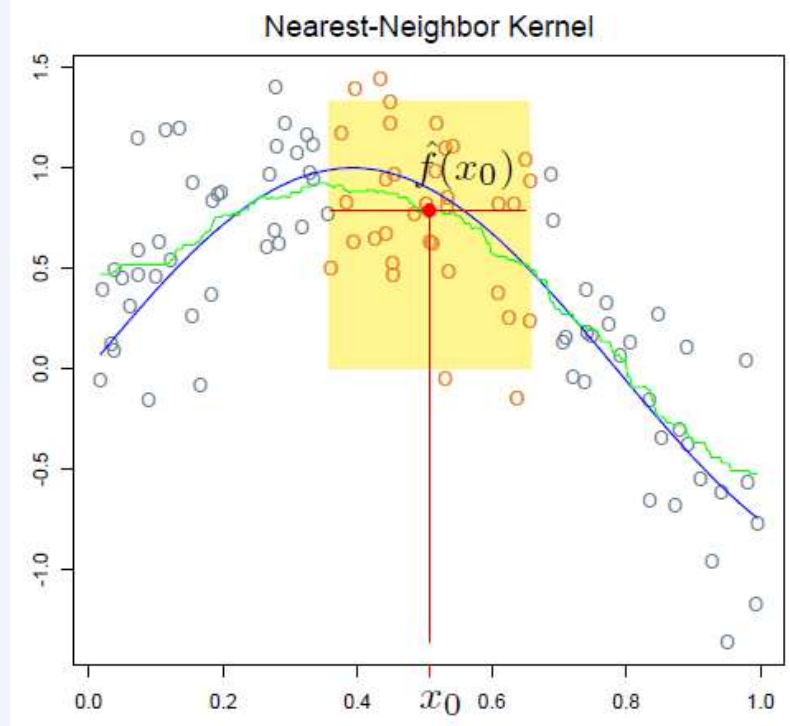
kNN: Kernel Regression

- What if we took all the points, not just the k nearest points, and introduced a weighting function that weights by distance (so that we weight the value of closer points more)?

$$\hat{y}^* = \frac{\sum_n K(x^*, x_n) y_n}{\sum_n K(x^*, x_n)}$$

- Traditional kNN method can be seen as a 0/1 weighting model
- Examples of kernels: Epanechnikov quadratic kernel, Tri-cube Kernel, and Gaussian Kernel.

Comparison of Kernels



Announcements

- Assignment 1 being graded, return to students by 8/28
 - Hopefully you understand the challenges in data collection and cleaning
 - Students need more practice on analysis and visualization
 - Didn't find anyone who got Playlist creation algorithm (Knapsack) right
 - We can have a deeper discussion on it on Piazza / Friday (8/28) evening
- Assignment 2 will be released 8/26, due 9/9
 - Will give practice on statistics and regression techniques
- Project Proposal template released, due 9/15
 - Those whose initial proposal was changed or need a discussion should attend office hours or set up an individual meeting with me

Lecture Objectives

- Understanding the outputs of a Linear Model
- Limitations of Linear Models and their extensions
- How to reduce over-fitting/variance via regularization
- Support Vector Machines (SVM)

Brief review of the linear model

$$y = f(x) + \varepsilon$$

response $\rightarrow$ *predictor* $\rightarrow$ *irreducible error*

If we assume that y is a linear combination of the predictor variables,

$$\hat{y} \cong \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \dots + \hat{\beta}_p x_p$$

We have n observations (training data);

$$y_1 \cong \hat{\beta}_0 + \hat{\beta}_1 x_{11} + \hat{\beta}_2 x_{12} + \hat{\beta}_3 x_{13} + \dots + \hat{\beta}_p x_{1p}$$

$$\underbrace{\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}}_Y \cong \underbrace{\begin{pmatrix} 1 & x_{11} & x_{12} & \dots & x_{1p} \\ 1 & x_{21} & x_{22} & \dots & x_{2p} \\ \vdots & \vdots & \vdots & \dots & \vdots \\ 1 & x_{n1} & x_{n2} & \dots & x_{np} \end{pmatrix}}_X \underbrace{\begin{pmatrix} \hat{\beta}_0 \\ \hat{\beta}_1 \\ \hat{\beta}_2 \\ \vdots \\ \hat{\beta}_p \end{pmatrix}}_B$$

Our goal
is to
minimize
 $\|y - XB\|_2$

Possible Questions

- How accurately do we know our model parameters?
- Is at least one predictor variable useful in the prediction?
 - We have to examine the p-values
- Which subset of the predictor variables are important?
 - There are several techniques of predictor variable/feature selection
- What would be the accuracy of predictions on unseen data?
 - We can generate confidence intervals on our estimates
 - Cross-validation gives us an estimate.
- Do I need more predictor variables/features?
 - Look at patterns in the residual errors

Confidence intervals for predictor estimators

- What causes errors in estimation of β_i ?
 - ε (noise in the model)
 - we do not know the exact form of $f(x)$
 - limited sample size
- Variance of $\hat{\beta}_i$ is called as **standard error**, $SE(\hat{\beta}_i)$
- To estimate SE, we use **Bootstrapping**
 - sampling from the training data (X,Y) to estimate its statistical properties.
- In our case, we can sample with replacement
 - Compute $\hat{\beta}_i$ multiple times by random sampling
 - Variance of multiple estimates approximates the true variance

Standard Errors Intuition from Formulae

- **Better model:** $(\hat{f} - y_i) \downarrow \Rightarrow \sigma \downarrow \Rightarrow SE \downarrow$

$$\sigma \approx \sqrt{\sum \frac{(\hat{f}(x) - y_i)^2}{n - 2}}$$

- **More data:** $n \uparrow$ and $\sum_i (x_i - \bar{x})^2 \uparrow \Rightarrow SE \downarrow$

- **Larger coverage:** $var(x)$ or $\sum_i (x_i - \bar{x})^2 \uparrow \Rightarrow SE \downarrow$

- **Better data:** $\sigma^2 \downarrow \Rightarrow SE \downarrow$

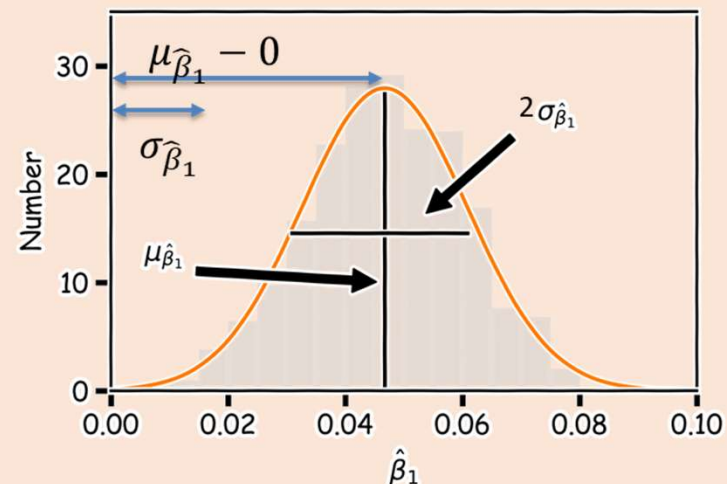
$$SE(\hat{\beta}_0) = \sigma \sqrt{\frac{1}{n} + \frac{\bar{x}^2}{\sum_i (x_i - \bar{x})^2}}$$
$$SE(\hat{\beta}_1) = \frac{\sigma}{\sqrt{\sum_i (x_i - \bar{x})^2}}$$

$$\text{General Formula: } SE(\beta)^2 = \sigma^2 (X^T X)^{-1}$$

Significance of predictor variables

- As we saw, there are inherent uncertainties in estimation of β
- We evaluate the importance of predictors using hypothesis testing, using the t-statistics and p-values (Small p-value(<0.05) => significant)
- Null hypothesis is that $\beta_i=0$

Test statistic here would be $t = \frac{\mu_{\hat{\beta}_1}}{\sigma_{\hat{\beta}_1}}$
Which measures the distance of the mean from zero in units of standard deviation.



Sample Results

```
import statsmodels.api as sm
est = sm.OLS(y, X2)
est2 = est.fit()
print(est2.summary())
```

OLS Regression Results						
Dep. Variable:	sales	R-squared:	0.897			
Model:	OLS	Adj. R-squared:	0.896			
Method:	Least Squares	F-statistic:	570.3			
Date:	Wed, 28 Nov 2018	Prob (F-statistic):	1.58e-96			
Time:	22:06:58	Log-Likelihood:	-386.18			
No. Observations:	200	AIC:	780.4			
Df Residuals:	196	BIC:	793.6			
Df Model:	3					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
const	2.9389	0.312	9.422	0.000	2.324	3.554
x1	0.0458	0.001	32.809	0.000	0.043	0.049
x2	0.1885	0.009	21.893	0.000	0.172	0.206
x3	-0.0010	0.006	-0.177	0.860	-0.013	0.011
Omnibus:	60.414	Durbin-Watson:	2.084			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	151.241			
Skew:	-1.327	Prob(JB):	1.44e-33			
Kurtosis:	6.332	Cond. No.	454.			
R ² , p-value and F-statistic						

Subset Selection Techniques

- Total number of subsets of a set of size $J = ?$
- **Goal:** All the variables in the model should have sufficiently low p-values, and all the variables outside the model should have a large p-value if added to the model.
- Three possible approaches
 - Forward selection
 - Backward selection
 - Mixed selection

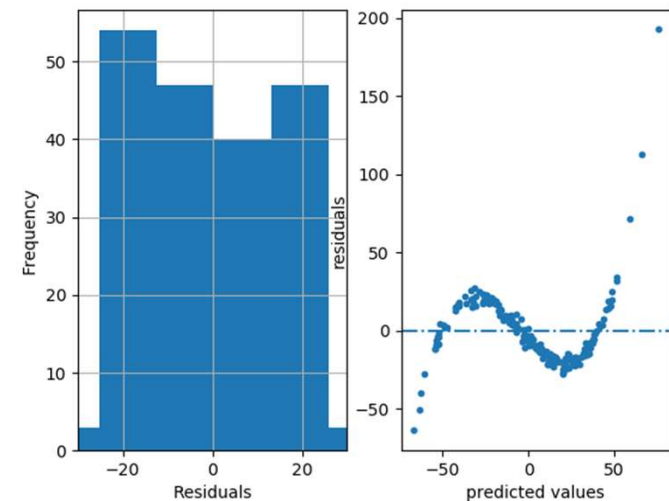
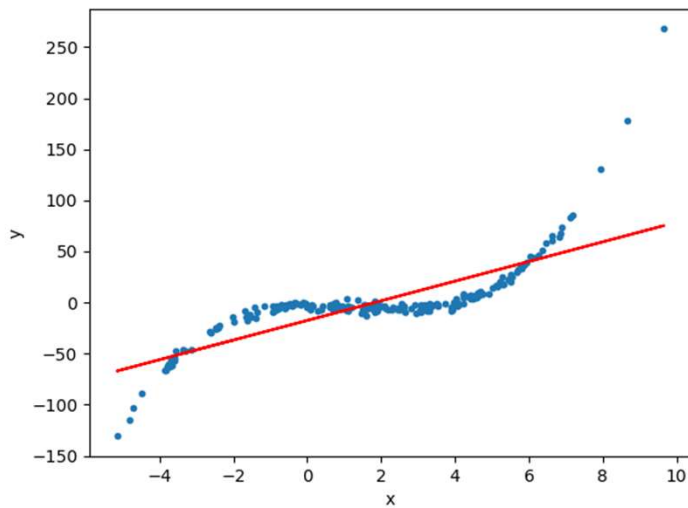
Subset Selection Techniques

- **Forward selection:**
 - Begin with a null set, S
 - Perform J linear regressions, each with exactly one variable
 - Add the variable that results in lowest Cross-validation error to the set, S
 - Again, perform $J-1$ linear regressions with 2 variables
 - Add the variable that results in lowest Cross-validation error to the set, S
 - Continue until some stopping criteria is reached... eg. CV error is not decreasing
- **Backward selection** begins with all the variables and removes the variable with highest p-value at successive steps
- **Mixed selection** is similar to Forward Selection, but it may also remove a variable if it doesn't yield any improvement to the model

Do I need more predictors/change of model?

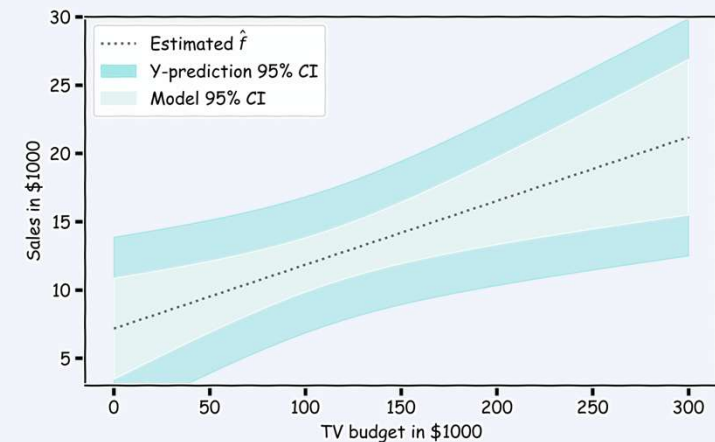
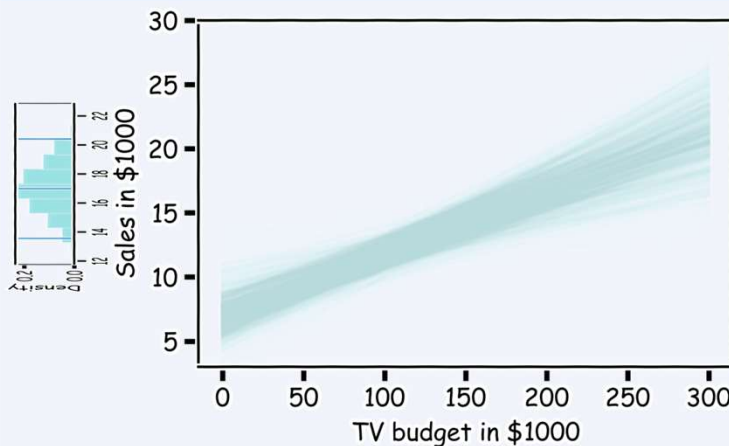
- When we estimated the variance of ϵ , we assumed that the residuals $r_i = y_i - \hat{y}_i$ were uncorrelated and normally distributed with mean 0 and fixed variance.
- These assumptions need to be verified using the data. In residual analysis, we typically create two types of plots:
 1. a plot of r_i with respect to x_i or $\hat{y}_i$. This allows us to compare the distribution of the noise at different values of x_i .
 2. a histogram of r_i . This allows us to explore the distribution of the noise independent of x_i or $\hat{y}_i$.

Patterns in Residuals



- Residuals are easier to interpret than the model
- We plot $(y_i - \hat{y}_i)$ with y_i , so the graph is always 2-D

Confidence intervals on predictions of y



- Depends on confidence on β
- Different $\beta \Rightarrow$ different values of y
- Given x , examine distribution of $\hat{f}$, determine the mean and standard deviation.
- For each of these $f(x)$, the prediction for $y \sim N(f, \sigma_\epsilon)$

Potential problems of Linear Models

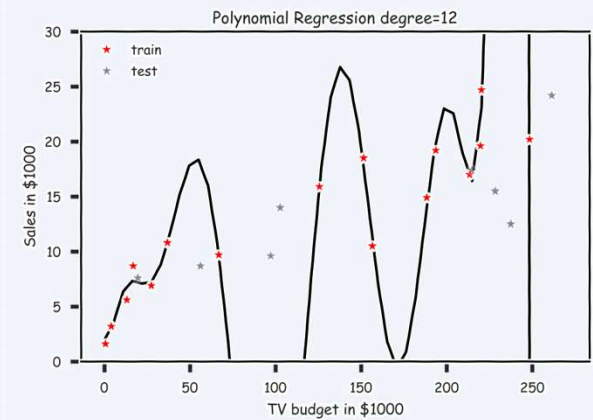
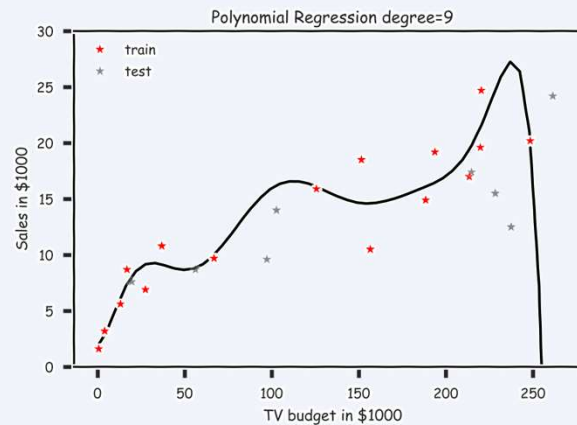
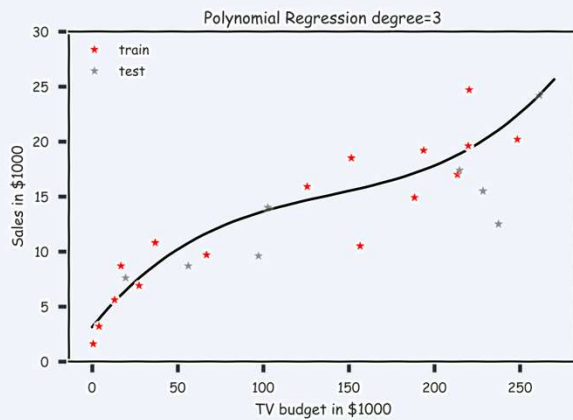
- Non-linearity
 - Can use polynomial linear regression or design better features
- Outliers
 - Disturbs the models because of quadratic penalty, Discard outliers carefully
- High-leverage points
 - Outliers in the predictor variables
- Collinearity (2 or more predictor variables have high correlation)
 - Keep one of them or design a good combined feature
- Correlation of error terms, Non-constant variance of error terms
 - Gives higher confidence in the model, can't trust the CI on model parameter

Polynomial Regression

- The simplest non-linear model we can consider, for a response Y and a predictor X , is a polynomial model of degree M , $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_M x^M + \epsilon$.
- Just as in the case of linear regression with cross terms, polynomial regression is a special case of linear regression - we treat each x^m as a separate predictor. Thus, we can write

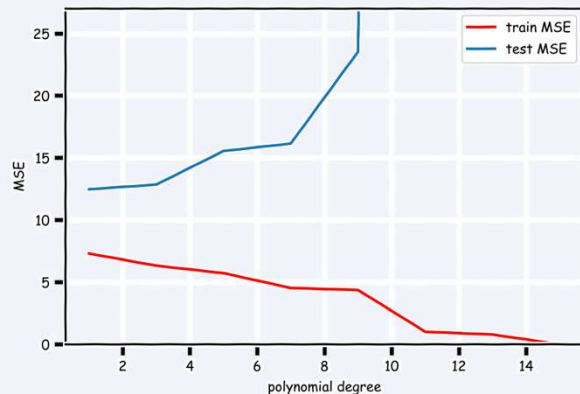
$$\mathbf{Y} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}, \quad \mathbf{X} = \begin{pmatrix} 1 & x_1^1 & \dots & x_1^M \\ 1 & x_2^1 & \dots & x_2^M \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & \dots & x_n^M \end{pmatrix}, \quad \boldsymbol{\beta} = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_M \end{pmatrix}.$$

Polynomial Regression



- Which of the above three is the best model?
 - Check RMSE
 - Check R^2
 - Remember bias and variance??

Benefit of Cross-Validation



$$CV(\text{Model}) = \frac{1}{K} \sum_{i=1}^K L(\hat{f}_{C-i}(C_i))$$

- Using cross-validation, we generate validate the models on a portion of training data which our learning algorithm has never seen.
- Leave-one out method is used when the number of sample points is very small.

Regularization of Linear Models

- Goal: Reduce over-fitting of the data by reducing degrees of freedom
- For a linear model, regularization is typically achieved by constraining the weights of the model

$$L_{reg}(\beta) = L(\beta) + \lambda R(\beta),$$

where λ is a scalar that gives the weight (or importance) of the regularization term.

- Fitting the model using the modified loss function L_{reg} would result in model parameters with desirable properties (specified by R).

Ridge Regression

- Alternatively, we can choose a regularization term that penalizes the squares of the parameter magnitudes. Then, our regularized loss function is:

$$L_{Ridge}(\beta) = \frac{1}{n} \sum_{i=1}^n |y_i - \beta^\top \mathbf{x}_i|^2 + \lambda \sum_{j=1}^J \beta_j^2.$$

- Works best when least-square estimates have high variance
- As λ increases, flexibility decreases, variance decreases, bias increases slightly

- Note that $\sum_{j=1}^J |\beta_j|^2$ is the l_2 norm of the vector $\boldsymbol{\beta}$ $\sum_{j=1}^J \beta_j^2 = \|\boldsymbol{\beta}\|_2^2$

Ridge Regression

- We often say that L_{ridge} is the loss function for l_2 regularization.
- Finding the model parameters β_{ridge} that minimize the l_2 regularized loss function is called **ridge regression**.

```
In [ ]: from sklearn.linear_model import Ridge
```

```
In [20]: X_train = train[all_predictors].values
X_val = validation[all_predictors].values
X_test = test[all_predictors].values

ridge_regression = Ridge(alpha=1.0, fit_intercept=True)
ridge_regression.fit(np.vstack((X_train, X_val)), np.hstack((y_train, y_val)))

print('Ridge regression model:\n {} + {}^T . x'.format(ridge_regression.intercept_, ridge_regression.coef_))
```

```
Ridge regression model:
-525.7662550875951 + [ 0.24007312  8.42566029  2.04098593 -0.04449172 -0.01227935  0.41902475
-0.50397312 -4.47065168  4.99834262  0.          0.          0.29892679]^T . x
```

```
In [21]: print('Train R^2: {}, test R^2: {}'.format(ridge_regression.score(np.vstack((X_train, X_val)),
                                                                    np.hstack((y_train, y_val))),
                                                                    ridge_regression.score(X_test, y_test)))
```

```
Train R^2: 0.5319764744847737, test R^2: 0.7881798111697319
```

LASSO (least absolute shrinkage and selection operator) Regression

- Ridge regression reduces the parameter values but doesn't force them to go to zero. LASSO is very effective in doing that.
- It uses the following regularized loss function is:

$$L_{LASSO}(\beta) = \frac{1}{n} \sum_{i=1}^n |y_i - \beta^\top \mathbf{x}_i|^2 + \lambda \sum_{j=1}^J |\beta_j|.$$

- Note that $\sum_{j=1}^J |\beta_j|$ is the l_1 norm of the vector β

$$\sum_{j=1}^J |\beta_j| = \|\beta\|_1$$

LASSO Regression

- Hence, we often say that L_{LASSO} is the loss function for l_1 regularization.
- Finding the model parameters β_{LASSO} that minimize the l_1 regularized loss

```
In [ ]: from sklearn.linear_model import Lasso
```

```
In [22]: lasso_regression = Lasso(alpha=1.0, fit_intercept=True)
lasso_regression.fit(np.vstack((X_train, X_val)), np.hstack((y_train, y_val)))

print('Lasso regression model:\n {} + {}^T . x'.format(lasso_regression.intercept_, lasso_regression.coef_))
```

Lasso regression model:
10.424895873901445 + [0.24482603 3.48164594 1.84836859 -0.06864603 -0. -0.
-0.02249766 -0. 0. 0. 0. 0.]^T . x

```
In [23]: print('Train R^2: {}, test R^2: {}'.format(lasso_regression.score(np.vstack((X_train, X_val)),
                                                                 np.hstack((y_train, y_val))),
                                                                 lasso_regression.score(X_test, y_test)))
```

Train R^2: 0.48154992527975765, test R^2: 0.6846451270316087

Choosing λ

- In both ridge and LASSO regression, we see that the larger our choice of the **regularization parameter** λ , the more heavily we penalize large values in β ,
- If λ is close to zero, we recover the MSE, i.e. ridge and LASSO regression is just ordinary regression.
- If λ is sufficiently large, the MSE term in the regularized loss function will be insignificant and the regularization term will force β_{ridge} and β_{LASSO} to be close to zero.
- To avoid ad-hoc choices, we should select λ using cross-validation.
- Once the model is trained, we use the unregularized performance measure to evaluate the model's performance.

Elastic Net

- Middle ground between Ridge and Lasso regression
- Regularization term is a simple mix with parameter 'r'

$$L_{EN}(\beta) = \frac{1}{n} \sum_{i=1}^n |y_i - \beta^T x_i|^2 + (1-r)\lambda \sum_{j=1}^J \beta_j^2 + r\lambda \sum_{j=1}^J |\beta_j|$$

- Elastic Net has better convergence features over Lasso.

```
Sklearn.linear_model import ElasticNet  
Elastic_net = ElasticNet(alpha=0.1, l1_ratio=0.5)
```

SVM (Support Vector Machines)

- Uses a different approach to regression
- Instead of thinking of the fit as a line, let us think of it as a channel
- Fit as many instances as possible on the channel while limiting the margin violations, (i.e. instances off the channel)
- Width of the channel is the hyper-parameter ' ϵ '
- Adding more training instances within the channel doesn't change the model parameters
- Hence these models are more robust against over-fitting

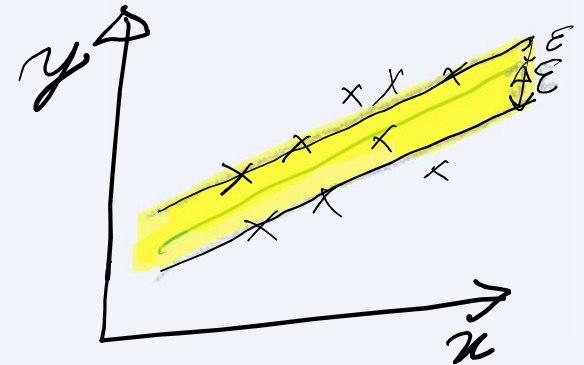
SVM Regression

Assume $f(x) = x^T \beta + \beta_0$
Let's minimize $H(\beta, \beta_0) = \sum_{i=1}^N V(y_i - f(x_i)) + \frac{\lambda}{2} \|\beta\|^2$

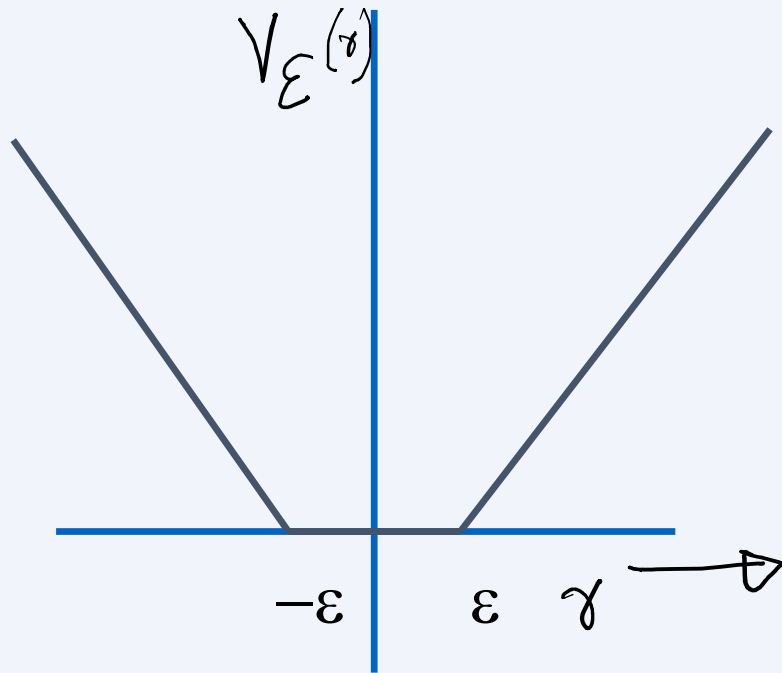
If we consider the Optimization Problem
minimize $\frac{1}{2} \|\beta\|^2$

$$\text{s.t. } \begin{cases} y_i - f(x_i) \leq \epsilon \\ f(x_i) - y_i \leq \epsilon \end{cases}$$

For a fixed ϵ , we may not have a feasible solution



ϵ insensitive Loss function



$$V_\epsilon(\rho) = \begin{cases} 0 & \text{if } |\rho| \leq \epsilon \\ |\rho| - \epsilon & , \text{ otherwise} \end{cases}$$

SVM Regression

So, let us allow some slack variables ξ_i & ξ_i^* (support vectors)

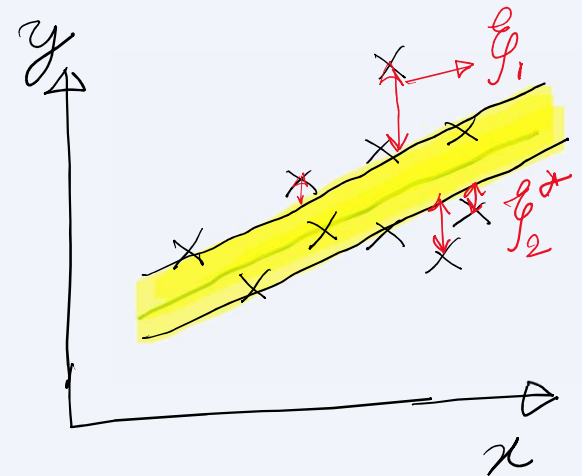
Consider the optimization problem:-

$$\text{Minimize} \quad \frac{1}{2} \|\beta\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*)$$

$$y_i - f(x_i) \leq \varepsilon + \xi_i$$

$$f(x_i) - y_i \leq \varepsilon + \xi_i^*$$

$$\xi_i \geq 0 \quad \xi_i^* \geq 0$$



```
Sklearn.svm import ElasticNet  
Svm_reg = LinearSVR(epsilon=1, C=2)
```

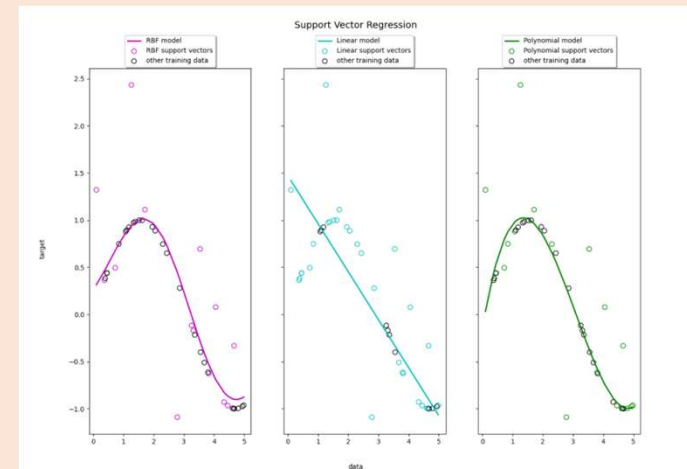
Parameters in SVM regression

- Parameter ϵ controls the width of the channel and can affect the number of support vectors used to construct the regression function.
- Adding more training vectors
- Bigger $\epsilon \Rightarrow$ fewer support vectors
- Parameter C determines the trade-off between the model complexity and the degree to which the deviations larger than ϵ can be tolerated
- It is interpreted as a traditional regularization parameter that can be estimated by Cross Validation, for example

Non-linear data

- SVM allow for a computationally efficient method of transforming the dataset to higher dimensions using *kernel trick*.
- Common kernels that are used are
 - Linear, polynomial, Gaussian RBF, Sigmoid

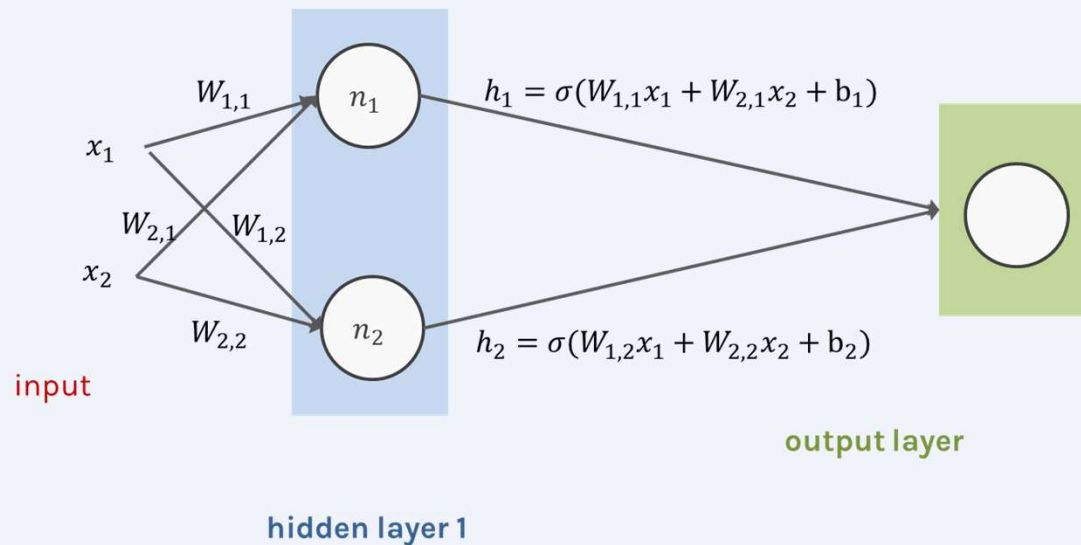
- linear: $\langle x, x' \rangle$.
- polynomial: $(\gamma \langle x, x' \rangle + r)^d$, where d is specified by parameter `degree`, r by `coef0`.
- rbf: $\exp(-\gamma \|x - x'\|^2)$, where γ is specified by parameter `gamma`, must be greater than 0.
- sigmoid $\tanh(\gamma \langle x, x' \rangle + r)$, where r is specified by `coef0`.



Neural Networks

- Use a multi-layer architecture with an input, output layer and several hidden layers of non-linear computational units called neurons
- They are able to compute complex functions over a range of inputs
- Initially the weights at each connection is assigned randomly
- A process called back-propagation is used to update these weights
- As long as there are a large number of training points, they can be generalized with high accuracy

Example of Neural Network



- This network has two inputs, one hidden layer
- σ is the activation function

Adding layers allows us to model increasingly complex functions

